Ministério da Educação
Universidade Federal de Itajubá
Campus Avançado de Itabira

Engenharia de Computação
ECOI2219 — Compiladores
Prof. Walter Aoiama Nagai

Laboratório 1

Tradução Dirigida por Sintaxe com PLY

Sprint 1	Liberado em 30/09 → Entrega em **14/10**, 23h55
Modalidade	Entrega em grupo (as mesmas equipes do Módulo 5)
Peso	10% da nota N2
Tempo estimado	12 a 16 horas de trabalho da equipe

Por que isto importa

Ao final deste laboratório, sua equipe terá escrito um **compilador que funciona**: programas na linguagem μC entram, código de máquina virtual sai. E não é um exercício solto — é o **mesmo compilador** que o projeto do curso usa, o que permite comparar seu resultado com o de referência, instrução por instrução.

Sumário

1 O que você vai construir **2**

2 Como este laboratório está organizado **2**
2.1 Os cinco passos . . . 2

3 Os três níveis de compreensão **3**

4 Ambiente **3**
4.1 O que instalar . . . 3

5 PLY em dez minutos **4**
5.1 Regras léxicas . . . 4
5.2 Regras sintáticas . . . 4

6 Passo 1 — O analisador léxico **5**
6.1 Traduzindo o lexer de referência . . . 5
6.2 Testando . . . 6

7 Passo 2 — Expressões **6**
7.1 O que o livro escreve . . . 6
7.2 O que você escreve . . . 7
7.3 Por que a ordem funciona . . . 7
7.4 Precedência . . . 7

8 Passo 3 — Declarações, atribuição e escrita **8**
8.1 Registrando a declaração . . . 8
8.2 Atribuição e leitura . . . 8

9 Passo 4 — O comando condicional **9**
9.1 O problema . . . 9
9.2 A solução: uma regra que reduz antes . . . 9
9.3 `if` com `else` . . . 10

10 Passo 5 — O laço **10**
10.1 O padrão . . . 10
10.2 `break` . . . 11

11 Executando no MEPA **11**
11.1 Gerar e executar . . . 11
11.2 Comparando com a referência . . . 12

12 Desafios **12**
12.1 Desafio 4.1 — `do-while` . . . 12
12.2 Desafio 4.2 — `read` e blocos . . . 12
12.3 Desafio 4.3 — Comparação estruturada com a referência . . . 12
12.4 Desafio 4.4 — Auditoria de código gerado por IA . . . 12

13 Entrega **13**
13.1 O relato técnico . . . 13

14 Política de uso de inteligência artificial **13**

15 Avaliação **14**

16 Para estudar mais **14**

1 O que você vai construir

Este laboratório reconstrói, em Python com a biblioteca PLY, a parte de **tradução dirigida por sintaxe** do compilador μC. A tradução dirigida por sintaxe é a ideia de que o próprio reconhecimento da gramática emite o código: não há uma etapa separada de "gerar código"; o código nasce enquanto a estrutura do programa é reconhecida.

A Figura 1 mostra onde estamos. Ao final do laboratório, sua equipe terá coberto as três primeiras caixas.

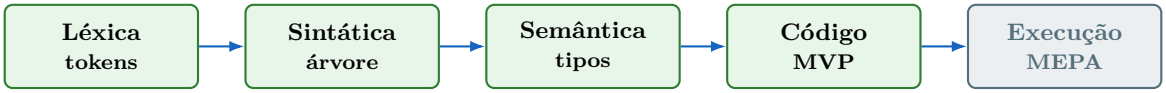


Figura 1: Pipeline de compilação do μC. As caixas verdes são o que este laboratório entrega; a caixa cinza é o interpretador que executa o resultado.

Antes de começar

Este laboratório assume três coisas. Se alguma delas não for familiar, retome aqui antes de seguir — o custo de recuperar agora é muito menor que o de travar no Passo 4.

- **Python básico**: funções, dicionários, classes simples.
- **Gramáticas livres de contexto** (Módulo 3): o que é uma produção, o que é uma derivação.
- **Expressões regulares** (Módulo 2): reconhecer padrões de texto.

Você não precisa saber PLY nem a linguagem μC de antemão: os dois são apresentados aqui.

2 Como este laboratório está organizado

O roteiro tem cinco passos, cada um acrescentando uma capacidade ao compilador. Os passos foram escritos para serem seguidos em ordem: cada um termina com um **checkpoint** — um resultado que você pode verificar antes de seguir. Se um checkpoint falhar, volte e conserte: os passos seguintes dependem dele.

Passo	O que acrescenta	Seção	Tempo
1	analisador léxico (reconhece tokens)	6	1–2 h
2	tradução de expressões	7	2–3 h
3	declarações, atribuição e escrita	8	2–3 h
4	comando condicional (`if`)	9	3–4 h
5	laço (`while`)	10	2–3 h
—	execução no MEPA e desafios	11	2–4 h

2.1 Os cinco passos

Passo 1 — O analisador léxico.
Transforma texto em uma sequência de tokens. É a parte mais mecânica e a mais imediata: você vê o resultado na hora.

Passo 2 — Expressões.
Emite código para somas, subtrações, multiplicações e divisões. Aqui aparece o conceito central: a ação semântica roda quando a produção *reduz*, e não quando o token é lido.

Passo 3 — Memória.
Introduz a tabela de símbolos e a noção de *deslocamento* (offset): onde cada variável mora na memória.

Passo 4 — Controle de fluxo.
O passo mais difícil e o mais importante. Você vai emitir desvios condicionais e descobrir o problema do "rótulo no meio".

Passo 5 — Laços.
Fecha o ciclo, reaproveitando o que o Passo 4 ensinou.

3 Os três níveis de compreensão

Cada passo tem três camadas de informação, visualmente distintas. Você pode escolher por onde começar, e pode voltar a cada uma quantas vezes precisar.

Caixa	Para que serve
Por que isto importa	Situa a etapa no problema maior. Leia se estiver se perguntando "onde vou usar isto?"
Antes de começar	Lista o que você precisa saber para aquele passo. Leia se travar por falta de base.
Sua vez	A tarefa prática. É o que você faz.
Armadilha	Um erro comum, já identificado. Poupam horas.
Você conseguiu?	Como saber que o passo funcionou antes de seguir.
Para pensar	Pergunta de reflexão. Não bloqueia o avanço.

Sua vez

Escolha onde guardar o trabalho. Sugestão de organização da pasta da equipe:

```
lab1_tds/
  lab1_tds.py # seu compilador
  exemplos/ # programas de teste (.uc)
  mepa/ # interpretador MEPA 5.0
  relato_lab1.md # relato tecnico do grupo
```

Todos os integrantes devem ter acesso de escrita ao repositório. Trabalhar em arquivos separados e juntar no final custa, tipicamente, mais tempo do que a coordenação imediata.

4 Ambiente

4.1 O que instalar

Item	Como verificar
Python 3.10 ou superior	`python3 --version`
PLY (biblioteca)	`python3 -c "import ply; print(ply.__version__)"`
Interpretador MEPA 5.0	pasta `mepa/` fornecida pela disciplina
Compilador μC (referência)	`uc-full/` no repositório da disciplina

Para instalar o PLY em um ambiente isolado:

```
python3 -m venv .venv
source .venv/bin/activate
pip install ply
```

Antes de começar

O PLY precisa que *todas* as regras estejam no mesmo módulo Python, ou que sejam passadas explicitamente. Se você dividir o lexer e o parser em arquivos diferentes, leia a seção "Using multiple modules" da documentação do PLY. Para este laboratório, a estrutura mais simples — um arquivo só — é suficiente e recomendada.

5 PLY em dez minutos

O PLY é uma implementação em Python das ferramentas clássicas `lex` e `yacc`. Ele gera, a partir das suas regras, dois programas: um analisador léxico e um analisador sintático LALR. Você escreve as regras; ele escreve o autômato.

A correspondência com o que você já estudou é direta:

Teoria (Módulos 2 e 3)	**Como se escreve no PLY**
Expressão regular de um token	`t_NOME = r'...'`
Ação ao reconhecer um token	`def t_NOME(t):`
Produção $A \to \alpha\beta$	docstring `"A : α β"`
Ação semântica da produção	corpo da função `p_*`
Tabela de análise LALR	gerada automaticamente

5.1 Regras léxicas

```python
import ply.lex as lex

tokens = ['NUM', 'ID', 'PLUS', 'TIMES']
literals = '{}();' # tokens de 1 caractere

t_ignore = ' \t' # o que descartar

t_PLUS = r'\+' # regra SIMPLES: so a expressao regular
t_TIMES = r'\*'

def t_NUM(t): # regra FUNCAO: quando ha acao
    r'\d+'
    t.value = int(t.value)
    return t

def t_newline(t):
    r'\n+'
    t.lexer.lineno += len(t.value)

def t_error(t): # tratamento de erro lexico
    print(f"caractere ilegal {t.value[0]!r} na linha {t.lexer.lineno}")
    t.lexer.skip(1)
```

Armadilha

A **ordem** importa. O PLY ordena as regras pelo tamanho da expressão regular, implementando o *maximal munch* (a regra do Módulo 2). Assim, `t_LE` (dois caracteres) vence `t_LT` (um) quando o texto é `<=`. Você não precisa ordenar à mão, mas precisa entender por que `<=` não vira `<` seguido de `=`.

5.2 Regras sintáticas

```python
import ply.yacc as yacc

start = 'program'

def p_program(p):
    """program : block"""
    p[0] = p[1] # p[0] = atributo SINTETIZADO

def p_expr_plus(p):
    """expr : expr PLUS term""" # producao: expr -> expr PLUS term
    p[0] = ('+', p[1], p[3]) # corpo = acao semantica
```

A convenção que você precisa memorizar é pequena:

Escrita	Significado
`p[0]`	valor do não-terminal que está sendo reduzido
`p[1] …p[n]`	os símbolos do corpo, na ordem em que aparecem
`p.lineno(k)`	linha do k-ésimo símbolo (útil em mensagens de erro)
docstring	a produção; use `|` para alternativas

Para pensar

No livro-texto, uma ação semântica pode ficar *no meio* da produção — antes de um símbolo ainda não lido. No PLY, não: a função inteira roda de uma vez, quando a produção reduz. Guarde essa diferença: ela é o assunto do Passo 4.

6 Passo 1 — O analisador léxico

Por que isto importa

Antes de traduzir, é preciso reconhecer. O analisador léxico é o único componente do compilador que "vê" o texto original; tudo o que vem depois trabalha com a sequência de tokens que ele produz. Se ele erra, todo o resto erra — e o erro é difícil de achar lá na frente.

6.1 Traduzindo o lexer de referência

O compilador μC tem um analisador léxico escrito à mão (`src/lexer/Lexer.java`). Ele registra as palavras reservadas assim:

```
keyWords(new Word("if", Tag.IF));
keyWords(new Word("else", Tag.ELSE));
keyWords(new Word("while", Tag.WHILE));
keyWords(new Word("print", Tag.PRINT));
keyWords(Type.Int);
```

No PLY, a mesma informação vira um dicionário:

```
reserved = {
   'if': 'IF', 'else': 'ELSE', 'while': 'WHILE',
   'break': 'BREAK', 'print': 'PRINT',
   'true': 'TRUE', 'false': 'FALSE',
   'int': 'INT',
}
tokens = ['ID', 'NUM'] + list(reserved.values())

def t_ID(t):
   r'[a-zA-Z_][a-zA-Z0-9_]*'
   t.type = reserved.get(t.value, 'ID') # tabela substitui 'words'
   return t
```

Sua vez

Complete as regras que faltam. Cada linha é uma expressão regular:

Token	Texto	Expressão regular
EQ	==	r'=='
NE	!=	(complete)
LE	<=	(complete)
GE	>=	(complete)
AND	&&	(complete)
OR	||	(complete)
NOT	!	(complete)
LT	<	(complete)
GT	>	(complete)
ASSIGN	=	(complete)

Nota: `=` e `==` começam igual. O *maximal munch* resolve — mas só se as duas regras existirem.

6.2 Testando

```
if __name__ == '__main__':
    lexer = lex.lex()
    lexer.input(open('exemplos/expr.uc').read())
    for tok in lexer:
        print(f"{tok.lineno:3} {tok.type:8} {tok.value!r}")
```

Para o trecho `int a; a = 1 + 2;` a saída esperada é:

```
  1 INT 'int'
  1 ID 'a'
  1 ';' ';'
  1 ID 'a'
  1 '=' '='
  1 NUM 1
  1 '+' '+'
  1 NUM 2
  1 ';' ';'
```

Você conseguiu?

✓ O lexer lista os tokens sem erro.
✓ As palavras reservadas (`int`, `if`) aparecem como `INT` e `IF`, não como `ID`.
✓ Números com parte decimal viram `REAL` e sem parte decimal viram `NUM`.
Se algo falhar: verifique se o nome da regra corresponde exatamente ao nome em `tokens` (maiúsculas), e se `t_ID` vem depois de `t_(palavra reservada)` — se você escreveu as reservadas como função.

7 Passo 2 — Expressões

Por que isto importa

Toda linguagem de programação é, no fundo, um avaliador de expressões. Se você entender como `1 + 2 * 3` vira quatro instruções de máquina, entendeu o essencial de um gerador de código. O resto é repetição com nomes diferentes.

7.1 O que o livro escreve

A definição dirigida por sintaxe clássica para expressões aritméticas é:

Produção	Regra semântica
$E \to E_1 + T$	$E.val = E_1.val + T.val$
$E \to T$	$E.val = T.val$
$T \to$ **digito**	$T.val =$ **digito**$.lexval$

Traduzindo: cada nó calcula seu valor a partir dos filhos. Isso se chama *atributo sintetizado* — a informação *sobe* pela árvore.

7.2 O que você escreve

No PLY, em vez de calcular um valor, você emite uma instrução:

```
def p_expr_plus(p):
    """expr : expr PLUS term"""
    emit("SOMA") # acao da producao E -> E + T
    p[0] = p[1] # o "valor" sobe

def p_expr_term(p):
    """expr : term"""
    p[0] = p[1]

def p_term_num(p):
    """term : NUM"""
    emit(f"CRCT {p[1]}") # constante literal
    p[0] = p[1]
```

7.3 Por que a ordem funciona

Esta é a parte que costuma confundir. Vamos traçar `a + 2` passo a passo.

Redução	Instrução emitida	Pilha depois
`factor : ID(a)`	`CRVL 0,0`	`[a]`
`factor : NUM(2)`	`CRCT 2`	`[a, 2]`
`expr : expr PLUS term`	`SOMA`	`[a+2]`

O `SOMA` sai por último porque a produção correspondente reduz por último. Se a ação estivesse "no meio" da produção, como no livro, o `SOMA` sairia antes dos operandos — e o MEPA receberia uma instrução sem nada na pilha.

Para pensar

A ordem de emissão é *pós-ordem* na árvore de derivação: filhos primeiro, pai depois. Por que isso é exatamente o que uma máquina de pilha precisa?

7.4 Precedência

A gramática do μC resolve precedência por *níveis*, como você viu no Módulo 3:

```
expr -> term (+ term | - term)* <- menor precedencia
term -> unary (* unary | / unary)* <- maior precedencia
```

Mantendo os níveis separados, o PLY resolve tudo pela estrutura, sem que você declare precedência.

Armadilha

Não escreva `expr : expr PLUS expr`. Essa produção é ambígua e o PLY reporta *conflito shift/-reduce*. O comportamento padrão (deslocar) dá associatividade à direita — errada para `+`. Use a gramática em níveis, como o compilador μC faz.

Sua vez

Acrescente `MULT` e `DIVI` à regra `term : term TIMES unary`. Depois compile `a + b * 3` e confira que `MULT` aparece antes de `SOMA`.

Você conseguiu?

✓ `a + b * 3` produz `MULT` e depois `SOMA`.
✓ `(a + b) * 3` produz `SOMA` e depois `MULT`.
✓ O programa executa no MEPA e imprime o valor correto.

8 Passo 3 — Declarações, atribuição e escrita

Por que isto importa

Até aqui as variáveis não existiam: você usava valores literais. Agora o compilador precisa lembrar *onde* cada variável foi declarada e *qual* o seu endereço — e essa memória auxiliar se chama tabela de símbolos.

8.1 Registrando a declaração

No μC, cada declaração faz duas coisas: registra o nome na tabela de símbolos e reserva memória.

```
// Parser.java, metodo decls()
Id id = new Id(tok, p, offset);
top.put(tok, id);
print("AMEM " + p.getWidth() + "\t; " + id.toString());
offset += p.getWidth();
```

A tradução:

```
def p_decl(p):
    """decl : INT ID ';'"""
    global OFFSET
    SYMBOLS[p[2]] = OFFSET # top.put(...)
    emit(f"AMEM 4\t; {p[2]} address: {OFFSET}")
    OFFSET += 4
```

Para `int a; int b;` a saída é:

```
    AMEM 4 ; a address: 0
    AMEM 4 ; b address: 4
```

Sua vez

Acrescente `float` à declaração. A largura é 8 bytes (consulte `symbols/Type.java`), então o `AMEM` e o incremento do offset mudam.

8.2 Atribuição e leitura

```
def p_stmt_assign(p):
    """stmt : ID ASSIGN bool ';'"""
    emit(f"ARMZ 0,{SYMBOLS[p[1]]}\t; {p[1]}")

def p_factor_id(p):
    """factor : ID"""
    if p[1] not in SYMBOLS:
        print(f"linha {p.lineno(1)}: variavel nao declarada")
        raise SystemExit(1)
    emit(f"CRVL 0,{SYMBOLS[p[1]]}\t; {p[1]}")
```

Armadilha

O offset é global, não por bloco. O compilador μC numera as variáveis em uma sequência única, mesmo quando há blocos aninhados. Se você reiniciar o offset a cada `{`, os deslocamentos ficam errados — e o programa *compila*, mas lê a memória errada. O único caso em que o μC reinicia o offset é dentro de *registros*.

Você conseguiu?

✓ `{ int a; float x; }` gera `AMEM 4` e `AMEM 8`.
✓ `a = 1 + 2;` compila e executa, imprimindo `3`.
✓ Uma variável não declarada produz mensagem de erro, e não uma falha do Python (`KeyError`).

9 Passo 4 — O comando condicional

Por que isto importa

Este é o passo mais importante do laboratório. Toda estrutura de controle — condicional, laço, `break` — depende da mesma ideia: emitir um desvio para um lugar que ainda não foi escrito. Entender isso aqui faz os passos seguintes parecerem triviais.

9.1 O problema

O padrão de tradução de um `if` é:

```
    traducao de bool
    DSVF L1 <- desvia se falso
    traducao de stmt
L1:
```

O problema é evidente: quando o `DSVF` é emitido, o rótulo `L1` ainda não existe. O corpo nem começou a ser traduzido.

Armadilha

A tentativa natural é esta — e ela **não funciona**:

```
def p_stmt_if(p):
    """stmt : IF '(' bool ')' stmt"""
    l1 = new_label()
    emit(f"DSVF {l1}") # tarde demais!
    emit(f"L{l1}:")
```

Quando esta função roda, o corpo *já foi* traduzido. O `DSVF` sai depois dele, o desvio nunca é executado e o `if` é simplesmente ignorado. O programa compila e roda — produzindo o resultado errado.

9.2 A solução: uma regra que reduz antes

Divida a produção em duas. A primeira reduz *depois* da condição e *antes* do corpo:

```
stmt : if_head stmt
if_head : IF '(' bool ')' <- reduz aqui
```

```
IF_PENDING = []

def p_if_head(p):
    """if_head : IF '(' bool ')'"""
    l1, l2 = new_label(), new_label()
    emit(f"DSVF {l1}") # agora sim: entre a condicao e o corpo
```

```
    IF_PENDING.append((l1, l2))

def p_stmt_if(p):
    """stmt : if_head stmt"""
    l1, l2 = IF_PENDING.pop()
    emit(f"L{l1}:") # rotulo depois do corpo
```

Por que isto importa

A pilha `IF_PENDING` é o que o livro chama de **atributo herdado**: a informação desce da regra "pai" para a regra "filha". Como as funções `p_*` não recebem um pai, o estado viaja em uma variável do módulo. Toda a mecânica de tradução dirigida por sintaxe em PLY se reduz a isso: *ou* o valor sobe por `p[0]`, *ou* desce por variáveis do módulo.

9.3 `if` com `else`

O mesmo padrão, aplicado duas vezes:

```
def p_stmt_if_else(p):
    """stmt : if_head stmt ELSE else_mark stmt"""
    emit(f"L{IF_PENDING.pop()[1]}:")

def p_else_mark(p):
    """else_mark : """
    l1, l2 = IF_PENDING[-1]
    emit(f"DSVS {l2}") # pula o else
    emit(f"L{l1}:") # chegada do then
```

Para pensar

Por que `IF_PENDING` é uma *pilha* e não uma variável simples? Pense no que acontece com `if (a) if (b) print(1);` — o segundo `if` precisa de rótulos próprios sem apagar os do primeiro.

Sua vez

Compile `if (a > 0) a = 1; else a = 2;` e execute no MEPA com `a = 5` e com `a = -5`. Verifique que os dois caminhos funcionam. Depois compare com o código que o compilador μC gera para o mesmo programa.

Você conseguiu?

✓ O `DSVF` aparece *entre* a condição e o corpo no código gerado.
✓ O `if-else` executa o ramo correto nos dois casos.
✓ `if` aninhado (`if (a) if (b) ...`) funciona.

10 Passo 5 — O laço

Por que isto importa

Um laço é um `if` cujo desvio de volta aponta para trás. Como o rótulo de início já existe quando o corpo termina, metade do problema do passo anterior desaparece. Mas aparece um novo: como o `break` sabe para onde pular?

10.1 O padrão

```
  L1: traducao de bool
      DSVF L2
      traducao de stmt
```

```
    DSVS L1
  L2:
```

O rótulo `L1` precisa existir *antes* da condição, então ele é emitido por uma regra que reduz no início:

```
WHILE_PENDING = []
LOOP_EXITS = [] # pilha de saida dos lacos (para o break)

def p_while_start(p):
    """while_start : """
    l1, l2 = new_label(), new_label()
    emit(f"L{l1}:") # inicio do laco
    WHILE_PENDING.append((l1, l2))
    LOOP_EXITS.append(l2) # break desvia para l2

def p_while_head(p):
    """while_head : WHILE '(' while_start bool ')'"""
    l1, l2 = WHILE_PENDING[-1]
    emit(f"DSVF {l2}") # sai se a condicao for falsa

def p_stmt_while(p):
    """stmt : while_head stmt"""
    l1, l2 = WHILE_PENDING.pop()
    LOOP_EXITS.pop()
    emit(f"DSVS {l1}") # volta para o teste
    emit(f"L{l2}:")
```

10.2 `break`

O `break` precisa saber o rótulo de saída do laço mais interno. A pilha `LOOP_EXITS` carrega essa informação — outro atributo herdado:

```
def p_stmt_break(p):
    """stmt : BREAK ';'"""
    if not LOOP_EXITS:
        err(f"linha {p.lineno(1)}: break fora de um laco")
    emit(f"DSVS L{LOOP_EXITS[-1]}\t; break")
```

Armadilha

O compilador de referência **valida** o `break` fora de laço, mas **não gera** o desvio correspondente. Compile um programa com `break` na IDE μC e observe: o `DSVS` não aparece no código, e o laço executa até o fim. Na sua implementação, gere o desvio — é uma melhoria documentada.

Você conseguiu?

✓ O laço `while (k <= 5)` imprime `1 2 3 4 5` no MEPA.
✓ `break` dentro do laço interrompe a repetição.
✓ `break` fora de laço é rejeitado com mensagem clara.

11 Executando no MEPA

Antes de começar

O MEPA 5.0 exige o rótulo e a instrução na *mesma linha*: `L1: CRVL 0,4`. O compilador μC escreve o rótulo em linha própria. Por isso existe um passo de conversão antes de executar. Se você emitir o rótulo já no formato do MEPA, esse passo deixa de ser necessário.

11.1 Gerar e executar

```
# 1. gerar o codigo
python3 lab1_tds.py exemplos/while.uc > mvpOutput.txt

# 2. converter rotulos (se necessario)
awk '/^[[:space:]]*L[0-9]+:[[:space:]]*$/ {p=$1; next}
    {if (p!="") {sub(/^[[:space:]]*/,""); print p" "$0} else print; p=""}' \
    mvpOutput.txt > mvp.mepa

# 3. executar
python3 mepa_pt.py --progfile mvp.mepa --silent
```

11.2 Comparando com a referência

Sua vez

Para cada programa de teste, compile com o seu compilador *e* com o compilador μC de referência. Compare as duas sequências de instruções, ignorando comentários e a linha `AMEM 4 ; temp (ponteiro)`, que é um detalhe do gerador Java. Aponte *todas* as diferenças e explique cada uma. *Numeração de rótulos pode diferir* legitimamente. O que importa é a semântica dos desvios: confirme executando os dois no MEPA.

12 Desafios

Os desafios são opcionais e independentes entre si, exceto onde indicado. Os dois últimos envolvem uso de IA em nível de auditoria.

12.1 Desafio 4.1 — `do-while`

O μC tem um comando pós-testado:

```
    do stmt while ( bool ) ;
```

Neste caso, o corpo vem *primeiro*, então não há o problema do rótulo no meio. Implemente e verifique que o corpo executa ao menos uma vez.

12.2 Desafio 4.2 — `read` e blocos

Complete o compilador com:

1. `stmt : READ '(' ID ')' ';'` com `LEIT` e `ARMZ`;
2. `stmt : block`, permitindo blocos aninhados.

   Critério: compilar `fib.uc`, executar no MEPA com entrada `10` e obter `10` e `55`.

12.3 Desafio 4.3 — Comparação estruturada com a referência

Escolha *um* programa da pasta `uc-full/examples/` e produza uma tabela de correspondência:

Produção	Regra no `Parser.java`	Regra `p_*` e instruções
`stmt : if ...`	`case Tag.IF`	`p_if_head`, `DSVF`, `L1:`
`expr : expr + term`	`while` em `expr()`	`p_expr_plus`, `SOMA`

12.4 Desafio 4.4 — Auditoria de código gerado por IA

Um modelo de linguagem propôs a regra abaixo para o `while`:

```
def p_stmt_while(p):
    """stmt : WHILE '(' bool ')' stmt"""
    l1 = new_label()
    emit(f"L{l1}:") # (A)
```

```
    emit(f"DSVF L{l2}") # (B)
    emit(f"DSVS L{l1}") # (C)
    emit(f"L{l2}:") # (D)
```

Sua vez

Audite o trecho:

1. Identifique **dois erros**. Dica: a variável `l2` nunca é criada, e a ordem de emissão está errada.
2. Explique por que o código *parece* plausível — a estrutura visual corresponde ao padrão do livro.
3. Escreva a versão correta e comprove executando no MEPA.

13 Entrega

Sua vez

Entregue via SIGAA até **14/10**, 23h55, em nome do grupo:

1. `lab1_tds.py` completo e comentado;
2. para cada programa de teste, o par `mvpOutput.txt` — o seu e o da referência;
3. `relato_lab1.md` (ver estrutura abaixo).

13.1 O relato técnico

O relato não é um formulário de respostas. É o registro do raciocínio da equipe, e é nele que a avaliação encontra a evidência de autoria. Um relato adequado contém:

Decisões
Para cada decisão relevante — a estrutura de dados, o padrão de regra auxiliar, o tratamento de erro —: o que se decidiu, quais alternativas foram consideradas e por que a escolhida venceu. *Uma decisão sem alternativa descartada é uma decisão não compreendida.*

Evidências
Os comandos executados e suas saídas. Comparações entre o seu código e o de referência, execuções no MEPA, testes que falharam antes da correção. *Afirmações sem saída reproduzível não são evidência.*

Divergências
Onde seu código difere do de referência, e por quê. Diferenças de numeração de rótulos são irrelevantes; diferenças de semântica de desvio não são.

Dificuldades
O que não funcionou, o que ficou incompleto, o que a equipe faria diferente. *Um relato sem dificuldades descreve um trabalho que não foi feito.*

Declaração de IA
Um parágrafo *por integrante*, conforme a Seção 14.

14 Política de uso de inteligência artificial

O laboratório usa a Escala AIAS com **níveis por etapa**. A razão é simples: há passos de descoberta de API, em que consultar uma ferramenta é produtivo, e há núcleos conceituais, em que consultar elimina justamente o aprendizado que se quer verificar.

Etapa	Nível e escopo
Passos 1 a 3	**2** — **Auxiliar**: consulta livre para mensagens de erro e recursos do PLY
Passos 4 e 5	**1** — **Sem IA**: o núcleo conceitual, resolvido pela equipe
Desafio 4.4	**4** — **Auditor**: a equipe audita código gerado por IA

A declaração é **individual**, ainda que a nota seja do grupo. Os núcleos marcados "1 — Sem IA" são os que a prova substitutiva individual cobra. O uso não declarado é tratado como questão educativa, conforme a política institucional.

15 Avaliação

Critério	Peso
Analisador léxico e sintático funcionais	40%
Geração de código correta e executável no MEPA	30%
Explicação do padrão de regra auxiliar	20%
Relato (decisões, evidências, dificuldades) e declaração AIAS	10%

16 Para estudar mais

AHO, A. V. et al.
Compiladores: princípios, técnicas e ferramentas. 2. ed. Pearson, 2008. Capítulos 5 (tradução dirigida por sintaxe) e 6 (geração de código).

BEASLEY, D.
PLY (Python Lex-Yacc). Documentação oficial: `https://www.dabeaz.com/ply/` — consulte especialmente as seções sobre `p_*`, `t_*`, `precedence` e `errorlog`.

LOUDEN, K. C.
Compiladores: princípios e práticas. Pioneira Thomson, 2004. Capítulo 5.

Código de referência
`uc-full/`: `Lexer.java` e `Parser.java`.

KOWALTOWSKI, T.
MEPA 5.0 — interpretador e especificação das instruções.

Prof. Walter Aoiama Nagai
ECOI2219 — Compiladores
`walternagai@unifei.edu.br`